Kiro Workflow Cheatsheet

Primeros Pasos • Setup del Proyecto • Estructura • Skills • Hooks • Specs • Flujos de Trabajo • Edición 2026

Fuente: Este documento fue elaborado en base a la documentación oficial de Kiro disponible en kiro.dev/docs. Para información actualizada, consultar siempre la fuente oficial.

◆ 1. Introducción a la Inteligencia Artificial Estructural

Kiro no es un autocompletador de código (como los copilotos tradicionales de 2023). Es un **Agente de Ingeniería Estructural Autónoma** que vive sincrónicamente en tu Terminal (CLI) y en tu Editor (IDE). Su propósito es planificar arquitecturas, leer tu repositorio matemáticamente usando Vectores (Embeddings), y editar archivos directamente asumiendo la responsabilidad end-to-end de una funcionalidad. Kiro razona **antes** de codificar.

◆ 2. Setup, Instalación y Entornos

Kiro opera en dos frentes simultáneos que comparten el mismo "cerebro" local: - **CLI (Terminal):** Ideal para servidores CI/CD, o desarrollo hardcodeado. Se instala vía `curl -fsSL https://kiro.dev/install.sh | bash`. - **IDE Extensión:** Ideal para refactors visuales y manejo de UI gráfica.

Puesta a punto inicial: Al entrar a tu repositorio limpio, ejecutás `kiro .` (o presionás `"Generate Steering Docs"` en la UI). Kiro descargará el AST (Abstract Syntax Tree), analizará tu Stack detectando librerías y armará la matriz de conocimiento en la carpeta oculta `.kiro/steering/`.

🎬 **Video Demo:** Panel de Kiro en acción

◆ 3. El Cerebro del Sistema: Steering Files

Los **Steering Files** (archivos `.md` alojados en `.kiro/steering/`) son la "Constitución" inquebrantable de tu proyecto. Filtran las alucinaciones de la IA prohibiéndole tomar decisiones arquitectónicas ajenas a tu negocio.

🎬 **Video Demo:** Configuración de Steering Files

Todo proyecto debe tener al menos estos 3 pilares fundacionales:

`tech.md` **(El Qué — Stack Tecnológico):**

```
# Stack Tecnológico
- Frontend: React 18 con Next.js 14 App Router (nunca Pages Router).
- Estado global: Zustand. Prohibido Redux o Context API.
- Estilos: Tailwind CSS v3. No usar CSS Modules.
- Backend: FastAPI con Python 3.11.
- ORM: Prisma. Todas las queries pasan por /src/services.
- Testing: Jest + React Testing Library. Cobertura mínima: 80%.
```

`product.md` **(El Por Qué — Core Business):**

```
# Producto: Sistema de Turnos Médicos
- Tipo de app: B2B SaaS para clínicas privadas.
- Usuarios finales: Médicos, secretarias y pacientes.
- Prioridades críticas: Privacidad de datos (HIPAA), accesibilidad AAA.
- Performance: Tiempo de carga < 2 segundos en 3G.
- El sistema NO debe exponer datos de un paciente a otro usuario bajo
  ninguna circunstancia.
```

`structure.md` **(El Cómo — Convenciones de Arquitectura):**

```
# Estructura del Proyecto
- /src/app → Rutas y páginas (Next.js App Router).
- /src/components → Componentes UI reutilizables.
- /src/services → Lógica de negocio y acceso a DB via Prisma.
- /src/lib → Utilidades compartidas (helpers, formatters).
- /prisma → Schema de base de datos y migraciones.
- Naming: camelCase para variables, PascalCase para componentes.
- Imports: Siempre usar alias '@/' en vez de rutas relativas '../'.
```

Steering Files Personalizados (Avanzado): No estás limitado a los 3 archivos base. Podés crear tantos Steerings como tu proyecto necesite. Ejemplo: un `api-design.md` que solo se cargue cuando editas archivos en `/src/api/`, o un `security-policy.md` que se

active manualmente cuando hagas auditorías. Kiro leerá el Frontmatter YAML de cada uno para decidir cuándo inyectarlo.

Auto-Generación Inteligente: Si tu proyecto ya existe y tiene código escrito, no necesitás redactar estos archivos a mano. Kiro puede escanear tu repositorio completo y auto-generar los Steerings fundacionales analizando tu `package.json`, tus imports, tu estructura de carpetas y tus patrones de naming. Solo presioná **"Generate Steering Docs"** y revisá lo que propone.

◆ 4. Gestión Sensible, Inclusión y Ceguera (`.kiroignore`)

No todo el contexto debe enviarse en cada Prompt. Hacerlo quema Tokens y marea al Agente.

Inclusion Modes (YAML Frontmatter): Podés determinar dinámicamente qué manual lee el bot agregando un encabezado a tus archivos `.md`:

```
---
inclusion: fileMatch
fileMatchPattern: ["src/api/**/*.ts"]
---
```

(Tipos de inclusión: `always` [Permanentemente atado], `auto` [Invocado vía NLP], `fileMatch` [Activado dinámicamente por la ruta editada], `manual` [Activado por tag #]).

El Escudo de Ceguera (`.kiroignore`): A diferencia de `.gitignore`, `.kiroignore` bloquea el acceso en **tiempo de lectura** del Agente. Si el bot intenta leer un certificado `.pem` o llaves AWS ocultas allí, recibirá un `HTTP 403 Access Denied` nativo, garantizando niveles de seguridad Enterprise inquebrantables. Por otro lado, `AGENTS.md` funciona como el estándar open-source de "Reglas pasivas" colocado en la raíz del repo para compartir contexto con agentes de terceros.

El Árbol de Estructura Visual del Proyecto: Para entender todas estas capas, así debe lucir el esqueleto maestro de un repositorio Kiro:

```
tu-proyecto/
├── .kiroignore             # Reglas para cegar al Agente (Ocultar .env y
secrets)
├── AGENTS.md               # Standard Open-Source para agentes IA
├── .kiro/
│   ├── steering/           # El Cerebro Semántico
│   │   ├── product.md
│   │   ├── tech.md
│   │   └── structure.md
│   ├── skills/             # Workflows On-demand (Guías MD + Scripts CLI)
│   ├── hooks.json          # Callbacks en background y seguridad
│   └── mcp-servers.json    # Conexiones externas (Bases de datos, AWS,
Github)
├── requirements.md         # Specs (Fase 1: Negocio y Aceptación con EARS)
├── design.md               # Specs (Fase 2: Arquitectura y Modelos DB)
└── tasks.md                # Specs (Fase 3: Ejecución atómica de Tareas
visual)
```

◆ 5. Metodologías de Ejecución: Vibe Mode vs Specs

Existen dos pedales de aceleración radicalmente opuestos para operar el Agente. Elegir el incorrecto puede resultar en código brillante pero desconectado de tu arquitectura, o en un sobrecosto de tokens inmanejable.

🎬 **Video Demo:** Spec Mode en acción

A. Vibe Mode (Modo Danza / Libre): Abris el chat de la extensión y charlás. Es un flujo puramente iterativo. - **Caso de Uso de Oro:** Prototipado rápido, dudas teóricas ("¿Me armás una RegEx rápida para un email?"), exploración de librerías nuevas o el scaffolding de un script aislado (Ej: un scraper en Python de 100 líneas). - **El Riesgo Oculto (Context Drift):** El Vibe Mode es un túnel de chat continuo. Si debatís un monolito durante 300 mensajes, el Agente comenzará a "alucinar" por deriva de contexto (olvidará las reglas de tu primer prompt, quemará enormes cuotas de tokens re-evaluando toda la charla histórica, y podría escupir código que rompa integraciones críticas).

B. Spec Mode (Desarrollo Quirúrgico Obligatorio): Este es el motor industrial de Kiro. Utilizado para interactuar con bases de código Enterprise existentes donde romper un contrato JSON cuesta horas de Debugging. Existen tres tipos de Specs: - **Feature Specs:** Construcción de piezas desde cero (Ej: "Módulo de Facturación"). - **Bugfix Specs:** Órdenes restrictivas para parchar problemas de producción. (El agente asume una personalidad cautelosa; tiene prohibido rediseñar o refactorizar código subyacente que

funcione, enfocándose sólo en el arreglo aséptico del Bug). - **Correctness Specs:** Orientados a validar y mejorar código existente sin agregar funcionalidades nuevas. Kiro analiza la lógica actual y verifica que cumpla con los contratos definidos, detectando edge cases no cubiertos, errores silenciosos y regresiones potenciales.

Estrategias de Arranque (Por Dónde Empezar): Kiro no te fuerza a un solo flujo lineal. Podés arrancar el Spec desde dos puntos de entrada según tu nivel de claridad sobre el problema:

- **Requirements-First (Top-Down):** El enfoque clásico. Arrancás definiendo *qué* necesita el negocio en `requirements.md` usando notación EARS. Kiro desciende desde la intención humana hacia la técnica: primero las historias de usuario, después la arquitectura, y finalmente las tareas de código. *Ideal cuando tenés claro el objetivo de negocio pero no sabés cómo implementarlo técnicamente.*
- **Design-First (Bottom-Up):** El enfoque del Arquitecto. Si ya sabés exactamente cómo debería verse la solución técnica (qué endpoints crear, qué tablas SQL agregar), podés arrancar directamente por `design.md`. Kiro toma tu plano arquitectónico y genera en reversa los Requerimientos y las Tareas. *Ideal cuando tenés el blueprint técnico pero necesitás que el bot atomice las balas de ejecución.*

¿La Magia del Spec Mode? Muta la conversación fluida hacia **Documentos Congelados y Aislados** (.md). El Agente tiene bloqueado tocar código fuente hasta que el Ingeniero pre-apruebe el diseño en fase de texto puro. Al hacer esto, la IA descarga de su memoria todo el debate basura previo, conservando únicamente las directivas estáticas puras, eliminando las alucinaciones al 100%.

◆ 6. El Protocolo EARS (Requirements Fase 1)

La primera fase de un Spec es el archivo `requirements.md`. En vez de dejar que el Dev escriba un párrafo ambiguo como "necesito un login", Kiro fuerza la redacción bajo **Notación EARS** (*Easy Approach to Requirements Syntax*), un estándar de ingeniería de software diseñado para eliminar huecos cognitivos y ambigüedades que generan bugs.

La Taxonomía Completa de EARS: EARS define 5 patrones concretos para redactar requerimientos inequívocos. Kiro los usa como plantilla interna:

- **Ubiquitous (Universal):** Reglas que aplican siempre, sin condiciones.

> *"El sistema debe cifrar todas las contraseñas usando bcrypt con un mínimo de 12 salt rounds."*

- **Event-Driven (Por Evento / Trigger):** Se activan cuando ocurre algo específico.

> *"Cuando el usuario efectúa el pago, entonces el sistema debe encolar la factura en el servicio de billing y enviar un email de confirmación."*

- **State-Driven (Por Estado):** Aplican mientras se mantiene una condición.

> *"Mientras el carrito de compras esté vacío, el sistema debe desactivar el botón de Checkout y ocultar el resumen de costos."*

- **Unwanted Behavior (Comportamiento No Deseado):** Definen qué debe pasar ante fallas o excepciones.

> *"Si el proveedor externo Stripe se cae por Timeout, entonces el sistema debe rechazar la transacción con HTTP 424, registrar el incidente en el log de errores y notificar al equipo de soporte vía Slack."*

- **Optional (Condicional por Feature Flag):** Aplican solo si una funcionalidad opcional está activa.

> *"Donde el módulo de analytics premium esté habilitado, el sistema debe trackear cada click del usuario en el dashboard y enviarlo a Mixpanel."*

Criterios de Aceptación (Acceptance Criteria): Dentro del `requirements.md`, Kiro no solo lista las historias. Para cada requerimiento, genera automáticamente **criterios de aceptación** verificables que sirven como contrato entre el Dev y el QA: - *"Given [contexto], When [acción], Then [resultado esperado]."* - Estos criterios son la fuente de verdad que Kiro usa después para auto-validar su propio código en la fase de Tasks.

El ingeniero siempre aprueba el Requirement antes de continuar a la fase de Design.

◆ 7. Diseño Arquitectónico y Task Engine (Fases 2 y 3)

Aprobado el requerimiento, pasamos al diseño: - `design.md`: Kiro dibuja la arquitectura interna, modelos SQL de base de datos y flujos de red. - `tasks.md`: La fase final. Kiro desglosa la funcionalidad entera en pequeñas "Balas atómicas" visuales (Checkbox).

El Ejecutor (▷): Dentro de `tasks.md`, verás cajas interactivas `[ ]`. Presionás Play. Kiro, trabajando en background asincrónico, abre el archivo fuente real, edita el código usando diffs semánticos, valida la sintaxis, y marca la tarea cómo `[X] Done`.

◆ 8. Agent Skills (Workflows On-Demand)

Las **Skills** no son simples prompts guardados. Adoptan el estándar abierto `agentskills.io`, permitiendo empaquetar flujos complejos de ingeniería (Instrucciones Markdown + Ejemplos de Código + Scripts Ejecutables) en módulos ultra-portables que Kiro invoca al vuelo.

La Anatomía de una Skill (`~/.kiro/skills/qa-auditor/`):

```
qa-auditor/
├── SKILL.md                # Archivo mandatorio. Contiene el ADN del
Workflow.
├── examples/               # Tu carpeta con plantillas 'Good_code.ts /
Bad_code.ts'.
└── scripts/
    └── validate_jwt.py     # Lógica ejecutable que la IA puede correr en
bash.
```

Auto-Invocación (Mecánica de NLP en Background): Kiro lee milisegundo a milisegundo el campo `description` alojado en el YAML basal de todas tus skills instaladas. Si le decís al agente: *"Necesito que lintees este código nuevo"*, Kiro puntúa tu necesidad mediante NLP (Natural Language Processing), detecta el *Match* con la skill `qa-auditor` y la **auto-activa** en las sombras anexando su `SKILL.md` entero al contexto antes de escribir una sola línea.

```
---
name: qa-auditor
description: Validaciones TypeScript estrictas (OBLIGATORIA al tocar APIs).
---
# Directriz de Auditoría
Prohibido usar tipos `ANY`. Usar Interfaces genéricas T obligatoriamente.
Tirar el script interno `python ./scripts/validate_jwt.py` luego de
refactorizar.
```

Ejemplo 2 — Skill de Containerización (`.kiro/skills/docker-deploy/SKILL.md`):

```
---
name: docker-deploy
description: Reglas para containerizar servicios Node.js y optimizar imágenes
Docker.
---
# Directrices Docker
- Usar multi-stage builds siempre. Etapa 1: build con node:20-alpine. Etapa
2: runtime con node:20-slim.
- Nunca copiar node_modules al contenedor. Ejecutar `npm ci --
only=production` en la etapa de runtime.
- Exponer puertos explícitamente con EXPOSE. No hardcodear env vars: usar ARG
o --env-file.
- Incluir HEALTHCHECK con curl al endpoint /health cada 30s.
```

Ejemplo 3 — Skill de Commits Estandarizados (`.kiro/skills/commit-messages/SKILL.md`):

```
---
name: commit-messages
description: Conventional Commits obligatorio. Invocar antes de commitear.
---
# Formato Requerido
Todos los commits deben seguir Conventional Commits:
- `feat(scope): descripción` para funcionalidades nuevas.
- `fix(scope): descripción` para correcciones de bugs.
- `refactor(scope): descripción` para cambios internos sin impacto funcional.
- `docs(scope): descripción` para cambios en documentación.
- El scope debe coincidir con el módulo afectado (auth, billing, ui).
- El cuerpo del commit debe explicar el "por qué", no el "qué".
```

Modos de Llamado Manual e Importación: - **Invocación Quirúrgica:** Podés forzarlas obligatoriamente desde la barra de Chat tipeando su nombre con Slash (`/qa-auditor`). - **Importación Global desde GitHub:** Las skills son compartibles. Todo tu equipo puede

clonar el Cerebro del Arquitecto Cloud usando CLI: `kiro skill import https://github.com/my-org/skills/tree/main/docker-master`

◆ 9. Seguridad Defensiva Local: Agent Hooks

Los **Hooks** (`.kiro/hooks.json`) son llamas a la acción (*Callbacks*) deterministas que interceptan el ciclo de vida de la Inteligencia Artificial. **Actúan como Guardias de Seguridad invisibles.**

🎬 **Video Demo:** Agent Hooks en acción

Interceptan Triggers (Ej: `FileSave` o `PreToolUse`).

```
"hooks": {
  "PreToolUse": [
    {
      "matcher": "Bash",
      "action": { "type": "command", "command": "scripts/security_check.sh",
"timeout": 5 }
    }
  ]
}
```

Si Kiro intenta ejecutar un borrado de disco en bash a ciegas, el Hook despierta y ejecuta `security_check.sh`. Si el Shell Script devuelve `Exit code: 0` (Sano), Kiro avanza. Si devuelve `Exit code: 2` (Pánico), la acción de la IA es violentamente bloqueada, protegiendo tu código local de estupideces.

◆ 10. Conectividad Externa: MCP (Model Context Protocol)

Por naturaleza, las IAs operan en una caja de contención y están ciegas a internet. El estándar abierto **MCP** le da a Kiro *"Ojos y Manos"* fuera del entorno de tu Editor. Configurando endpoints en `mcp-servers.json`, el Agente puede: - Leer ramas remotas completas usando la API oficial de GitHub. - Googlear la documentación de Prisma que salió hace 3 días usando la Search API de Brave. - Ejecutar queries lógicos SQL a tu base local PostgreSQL para entender la data *real* antes de codear.

◆ 11. Powers: Inyección de Sabiduría sobre MCP

Un servidor MCP te otorga "el botón ciego" (Ej: Conexión a Base de Datos). Pero un **POWER** encapsula esa herramienta *junto con un extenso Manual Subjetivo de Instrucciones* de tu empresa. En vez de tirarle el MCP a Kiro para que opere (arriesgándote a un *Drop Table*), instalás un Power. El Power le indica dinámicamente al agente la anatomía estricta de tu esquema particular, qué consultas GraphQL le tenés prohibidas usar en local y qué lógicas de paginación tenés documentadas corporativamente para ese MCP exacto.

Install Powers vs Create Powers: Existen dos flujos para incorporar Powers en tu proyecto: - **Install Powers (Preconstruidos):** Kiro trae un directorio integrado de Powers verificados y listos para usar. Podés explorarlos desde el panel lateral del IDE o vía CLI. Son paquetes cerrados y auditados (Ej: `GitHub Power`, `PostgreSQL Power`, `Brave Search Power`) que ya contienen la configuración MCP + las instrucciones humanas optimizadas. Solo necesitás proveer tus credenciales o tokens de acceso. - **Create Powers (Personalizados):** Si tu equipo opera con una API interna o un SaaS propietario (Ej: tu propio microservicio de facturación), podés crear un Power custom desde cero. Esto implica definir la conexión MCP manualmente y redactar el archivo de instrucciones que le enseña a Kiro cómo comportarse con esa herramienta específica.

Anatomía de un Power: Un Power tiene dos componentes indivisibles: 1. **Servidor MCP configurado:** El endpoint técnico (URL, puerto, token Auth) que abre el canal de comunicación real. 2. **Archivo de Instrucciones (`.md`):** El cerebro operativo. Un manifiesto Markdown que le enseña al Agente: - Qué datos puede leer y cuáles NO. - Qué operaciones destructivas tiene estrictamente prohibidas. - Cómo paginar resultados grandes para no saturar el contexto. - Qué convenciones de naming o schemas respetar.

¿Por qué no alcanza con MCP solo? Conectar un MCP desnudo es como darle las llaves de tu casa a un desconocido. El Power actúa como el contrato legal que acompaña esas llaves: le dice exactamente qué habitaciones puede entrar, qué cajones tiene prohibido abrir, y bajo qué circunstancias debe pedir autorización explícita antes de actuar.

◆ 12. Contexto Multihilo: Subagentes, Tangents y Checkpoints

Para no destruir el chat o llenarlo de basura conceptual, Kiro emplea 3 mecánicas multihilo: - **Tangent Mode:** Si en pleno diseño arquitectónico tenés la duda rápida *"¿Por qué Promise.all falla rápido?"*, abrís una burbuja tangencial (Sub-Chat), discutís la teoría pura en aislamiento, la cerrás, y tu chat principal queda limpio sin haberse contaminado de contexto basura ajeno a tu funcionalidad. - **Checkpoints:** Kiro guarda snapshots mentales en árbol. Si el bot rompió 5 archivos, abrís el registro temporal cognitivo del Chat, hacés *"Rewind"* hacia el punto previo, y Kiro deshace silenciosamente y a nivel atómico todos los códigos arruinados. - **Subagentes:** Kiro paraleliza tareas en `tasks.md` spawneando hilos ("mentes secundarias") que clonan el AST, trabajando de fondo con contextos propios separados de tu chat visual.

◆ 13. Auto-Diagnósticos: Copilot Terminal & Language Server

- **Language Server nativo:** Si usás TypeScript, Pylance, o GoPLs, los subrayados rojos (Squigglies) viajan silenciosas por un túnel directo hacia el cerebro Kiro. No hace falta copiar los errores jamás: Kiro ya sabe exactamente dónde te equivocaste con el Typo.
- **Terminal Copilot:** Ejecutaste un build (`npm run lint`). La consola de Linux escupe un muro gigante rojo horrendo de errores de dependencias. Presionás "Autopilot" y Kiro analiza matemáticamente el `stderr` detectando por red neuronal *exactamente* el driver de Webpack fallido para instalarte la solución de parche por vos.

◆ 14. Selección de Modelos Cognitivos Inteligentes (Model Agnostic)

Kiro no depende de un único proveedor de IA. Su arquitectura "Model Agnostic" soporta Switching dinámico mid-flight para optimizar respuestas, tokens y performance según la dificultad temporal de la tarea.

La Familia Claude: - **Claude 3.5 Opus:** Elevada lógica temporal y amplitud analítica. Altamente recomendado para iniciar el Feature Spec y crear Requerimientos (donde equivocarse duele mucho). - **Claude 3.5 Sonnet:** La bestia hiper-veloz. Recomendada cuando le das el Play (▷) en `tasks.md` para vomitar código iterativo (Vibe Mode).

Otras Alternativas de Inteligencia Soportadas: - **OpenAI (GPT-4o / GPT-4.5):** Opciones robustas para razonamiento matemático avanzado, scripting complejo o cuando tu ecosistema empresarial ya cuenta con SLAs y límites definidos junto a Microsoft/OpenAI. - **Google Gemini (1.5 Pro):** Insuperable para absorber ventanas de contexto colosales (hasta 2 millones de tokens). Ideal si necesitás que el Agente procese de un solo golpe documentaciones enteras en PDF, bases de datos masivas o logs de producción infinitos. - **Modelos Locales / Open-Source:** Kiro también permite integraciones hacia endpoints customizados para operar con modelos como *DeepSeek* o *Llama 3*, garantizando compliance militar donde el código jamás abandona tus servidores.

◆ 15. Referencia Rápida: Proveedores Dinámicos y Atajos

Inyectá flechas semánticas en vez de copiar/pegar documentos al chat Kiro.

SINTAXIS	IMPACTO EN EL AST
`#codebase`	Manda Inteligencia Vectorial RAG (Embeddings) a rastrear cruces de dependencias crudas en los miles de archivos de tu repositorio y los indexa.
`#file` o `@`	Inyecta un documento texto puntual (`@src/main.ts`) para focalización láser inmersiva de contexto.
`#terminal`	Inyecta todo el último bloque log asqueroso que escupió BASH en tu stdout para auto-corregimiento IA.
`#[[file:...]]`	Inyecta y hardcodea enlaces vivos de código dentro de tus Steering Docs ocultos. Siempre leen la versión actualizada en disco de tu archivo para que los manuales no queden obsoletos.
`/mi-skill`	Manda Auto-Evaluar a Kiro bajo los comandos paramétricos predescritos de algún paquete YAML específico tuyo.
`Cmd + Shift + P`	Abrir la UI Visual de Hooks en la paleta de comandos.
`kiro task run`	Correr el pipeline interactivo y automático de Specs desde CLI.

◆ 16. Billing y Planes de Suscripción

Kiro ofrece distintos niveles de acceso según las necesidades del desarrollador o la organización:

Plan Gratuito (Free Tier): - Acceso completo al IDE y CLI con funcionalidades core (Steering, Specs, Chat). - Límite mensual de interacciones con el Agente (tokens consumidos). - Ideal para desarrolladores individuales explorando la plataforma o en proyectos personales.

Plan Pro (Individual): - Límites expandidos de uso mensual. - Acceso prioritario a modelos de mayor capacidad (Opus). - Soporte para múltiples workspaces simultáneos. - Powers y Skills premium del directorio oficial.

Plan Enterprise / Organizaciones: - Administración centralizada de licencias para equipos. - Controles de seguridad avanzados (VPC Endpoints, SSO corporativo vía SAML/Okta). - SLAs de disponibilidad garantizados. - Auditoría y compliance de uso de tokens por usuario. - Soporte técnico dedicado.

Upgrade y Downgrade: Podés escalar o reducir tu plan en cualquier momento desde la configuración de tu cuenta en el IDE (`Settings > Billing`) o vía el portal web de Kiro. Los cambios se aplican en el próximo ciclo de facturación. Si hacés un downgrade, conservás el acceso Pro hasta que termine tu período actual sin perder ninguno de tus Steerings, Skills o Hooks configurados.

◆ 17. Troubleshooting (Resolución de Problemas Comunes)

macOS: "Kiro is damaged and can't be opened": Este pop-up es un **falso positivo** del Gatekeeper de macOS. Solución rápida: 1. Andá a **System Settings → Privacy & Security** y presioná **Allow** o **Open Anyway**. 2. Si persiste, ejecutá en la Terminal:

```
sudo xattr -d com.apple.quarantine /Applications/Kiro.app
```

Falla de Redirect en el Login (Autenticación): Si Kiro no abre el navegador al intentar loguearte: - **macOS:** Abrí `Help → Toggle Developer Tools → Console` y verificá que `ioreg` esté accesible en tu `$PATH`. - **Windows:** Corré Kiro con `--enable-logging` desde consola y verificar permisos de Administrador.

AWS IAM Identity Center: - Requiere una suscripción activa de **Q Developer Pro**. Si ves *"There was an error signing you in"*, verificalo. - Kiro usa **US East (N. Virginia)** por defecto. Si tu perfil está en otra región, usá Builder ID, Google o GitHub. - Las sesiones expiran por defecto a las 8 horas (configurable por Admin).

Shell Integration Unavailable: 1. **Actualizá Kiro:** `Cmd+Shift+P` → `Kiro: Check for Updates`. 2. **Habilitá la integración:** `Cmd+Shift+P` → `Kiro: Enable Shell Integration`. 3. **Si falla,** agregá manualmente a tu `~/.zshrc`:

```
[[ "$TERM_PROGRAM" == "kiro" ]] && . "$(kiro --locate-shell-integration-path
zsh)"
```

Kiro se queda en "Working..." (Powerlevel10k / Oh My Posh): Si usás Powerlevel10k, agregá a `~/.p10k.zsh`:

```
typeset -g POWERLEVEL9K_TERM_SHELL_INTEGRATION=true
```

Servidores MCP que no Conectan: 1. Verificá el estado del servidor en el panel lateral de Kiro (icono de estado). 2. Validá que `mcp.json` tenga sintaxis JSON correcta y que los comandos de arranque existan. 3. Revisá los logs en `Output Panel → "Kiro – MCP Logs"`.

Windows: Updates Disabled / OneDrive Path Issue: - Si Kiro fue instalado por un Admin, reinstalá como usuario normal para restaurar auto-updates. - Si tu perfil vive en OneDrive, mové el workspace de Kiro a una ruta local fuera de OneDrive.

Ayuda Adicional: - GitHub Issues → - Documentación Oficial →

> Bienvenido al futuro. Bienvenido a Kiro.

◆ Índice General

1. Introducción a la Inteligencia Artificial Estructural
2. Setup, Instalación y Entornos
3. El Cerebro del Sistema: Steering Files
4. Gestión Sensible, Inclusión y Ceguera (`.kiroignore`)
5. Metodologías de Ejecución: Vibe Mode vs Specs
6. El Protocolo EARS (Requirements Fase 1)
7. Diseño Arquitectónico y Task Engine (Fases 2 y 3)
8. Agent Skills (Workflows On-Demand)
9. Seguridad Defensiva Local: Agent Hooks
10. Conectividad Externa: MCP (Model Context Protocol)
11. Powers: Inyección de Sabiduría sobre MCP
12. Contexto Multihilo: Subagentes, Tangents y Checkpoints
13. Auto-Diagnósticos: Copilot Terminal & Language Server
14. Selección de Modelos Cognitivos Inteligentes (Model Agnostic)
15. Referencia Rápida: Proveedores Dinámicos y Atajos
16. Billing y Planes de Suscripción
17. Troubleshooting (Resolución de Problemas Comunes)